

Victoria Sheff

06/21/2026

CS 320

During Project One, I developed and tested three backend services: `ContactService`, `TaskService`, and `AppointmentService`. My approach to unit testing focused on verifying object creation, validating input requirements, and ensuring that service methods behaved correctly. I used JUnit 5 to create tests that covered both valid and invalid scenarios.

For the `Contact` service, I tested the ability to add contacts and prevent duplicate IDs. In the `testAddContact()` method, I verified that a valid contact could be added without exceptions by using `assertDoesNotThrow()`. In the `testDuplicateContact()` method, I confirmed that attempting to add a second contact with the same ID resulted in an `IllegalArgumentException`. I also verified that the `Contact` object stored information correctly by checking values returned from the getter methods.

For the `Task` service, I tested both the `Task` class and `TaskService` class. The `Task` constructor validates task IDs, names, and descriptions. I created tests to verify that IDs longer than ten characters, names longer than twenty characters, and descriptions longer than fifty characters generated exceptions. In the `TaskService` tests, I verified that tasks could be added successfully, duplicate task IDs were rejected, and task information could be updated correctly. After updating a task, I used `assertEquals()` to confirm that the new name and description were stored properly.

For the `Appointment` service, I tested constructor validation rules. The `Appointment` class ensures that appointment IDs are not null and do not exceed ten characters. It also verifies that appointment dates are not in the past and that descriptions do not exceed fifty characters. In the

testValidAppointmentCreation() method, I verified that valid appointments were created correctly. In the testInvalidAppointmentId() method, I confirmed that IDs exceeding ten characters produced an exception.

My testing approach aligned closely with the software requirements because each requirement was represented by at least one test case. Validation rules in the constructors and service classes were directly reflected in the JUnit tests. By testing both valid and invalid scenarios, I ensured that the requirements were fully addressed.

The quality of my JUnit tests was demonstrated through the coverage of both expected behavior and error conditions. I used assertions such as assertDoesNotThrow(), assertThrows(), and assertEquals() to verify that the classes behaved correctly. Testing boundary conditions, including maximum field lengths and duplicate IDs, increased confidence in the reliability of the code.

Writing JUnit tests helped me ensure that the code was technically sound. For example, in the TaskService tests, I used assertEquals("Project", task.getName()) and assertEquals("Updated description", task.getDescription()) to verify that updates occurred correctly. In the Appointment tests, I used assertions to confirm that values returned by getter methods matched the expected results.

I also ensured efficiency by using similar testing patterns across the Contact, Task, and Appointment services. This consistency reduced redundancy and made the tests easier to maintain. Organizing tests into separate classes for each service improved readability and simplified troubleshooting.

The primary software testing technique was unit testing. Unit testing focuses on testing individual classes and methods independently. This technique allowed me to verify that each class functioned correctly without relying on other components. I also employed boundary value testing. Boundary testing focuses on values that lie at the limits of requirements.

Other testing techniques that were not used include integration testing and system testing.

Integration testing examines how multiple components interact with each other, while system testing evaluates the complete application. Different testing techniques serve different purposes.

Unit testing is useful during development because it identifies defects early and simplifies debugging. Integration testing is important when verifying communication between components, while system testing provides confidence that the entire application functions correctly in a production environment.

I adopted a cautious and detail oriented mindset. I understood that small errors in validation logic could create larger problems later. It was important to appreciate the complexity and relationships within the code because the service classes depended on proper validation. A missed requirement could allow invalid data to enter the system and create unexpected behavior. Testing negative cases, such as duplicate IDs and invalid field lengths, helped ensure that these problems were prevented.

I also tried to limit bias while reviewing my code. Instead of assuming the code worked correctly, I intentionally wrote tests that challenged the implementation. For example, I created tests that expected exceptions to occur when invalid values were provided. This approach helped reduce confirmation bias and increased confidence in the reliability of the software.

Bias can be a concern when developers test their own work because they may naturally focus on successful scenarios rather than potential failures. Writing both positive and negative test cases helped me evaluate the software more objectively.

Maintaining discipline and commitment to quality is essential for software engineering professionals. Cutting corners in testing can introduce defects and increase technical debt, making future maintenance more difficult. I avoided technical debt by creating tests for every major requirement and ensuring that invalid inputs were handled appropriately. In the future, I plan to continue writing tests alongside development to maintain high quality and reliable software.

Project One strengthened my understanding of software testing and quality assurance. Through the use of JUnit 5, I was able to verify that the Contact, Task, and Appointment services satisfied their requirements and behaved correctly under both normal and abnormal conditions. This experience reinforced the importance of unit testing, boundary testing, and maintaining a disciplined approach to software quality.